

Quantisation:

Process of mapping continuous infinite values to a smaller set of discrete values

Bit more "Working" definition.

Given a set $X \subseteq \mathbb{R}$ of values and P as the probability distribution of X the function Q is quantisation function that maps X to a smaller range of values $|Y|$

$$Q: X \rightarrow \{0, \dots, |Y|-1\}$$

Is quantisation function.

$$Q^{-1}: \{0, \dots, |Y|-1\} \rightarrow \mathbb{R}$$

Stand alone, often we care about data reconstruction.

So

$$\text{Quantisation Error} = \mathbb{E}_{x \sim p} \left[\|x - \hat{\Phi} \Phi(x)\|_2^2 \right]$$

i.e MSE loss of reconstruction.

Q: Should we care always care about reconstruction loss?

A: No! it depends on how the data being quantised is used.

for example if we are quantising W , then

In the world of Computers, the values have to be quantised naturally.

So how do we represent numbers in

a Computer?

(A) Integer formats.

INT32 Max value: $2^{31} - 1$
Min value: -2^{31}
(
1 sign bit

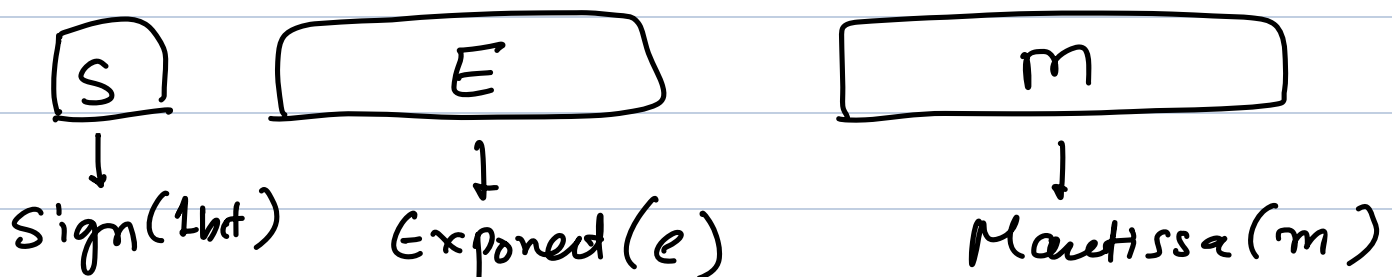
Positive numbers are stored as usual
-ve numbers as 2's complement

Avoids two zero problem ($+0, -0$)

(B) Floating point values. -

FP16, FP32, FP64,

↪ EeMm format.



Q. What does the $(1 + e + m)$ bit string represent?

Normal floating point numbers: $(E \neq 0)$
 E is not all 1s

Let $n(e)$: number represented by binary representation

$$V = (-1)^{n(s)} \cdot 2^{n(E) - \text{bias}} \cdot (1 + n(O.M))$$

$$\text{bias} = 2^{n(E)-1} - 1$$

Subnormal & Zero $(E = 0)$

$$V = (-1)^{n(s)} \cdot 2^{1-B} \cdot n(O.M)$$

Special numbers $(E = 1111 \dots)$

$$M = 0 \quad +\infty / -\infty$$

$M \neq 0$

NaN

[Add figure fp16-2.png]

* This is old convention. Nvidia has changed how we interpret the bits.

FP32, BF16, FP16, INT8, etc have different precision and Range

[Add dtypes image]

Precision = distance between different represented points

Range = max-min value that we can have.

Thus even numbers that we

Usually use quantises the ~~Real~~ Real line.

[Add fp8 image]

Quantisation in ML

FP32, FP16 are Common go-to-formats in ML. So in ML we care about going towards smaller bit widths, example

FP16, FP32 $\longrightarrow$ FP8, INT8
INT4

Example: $X \sim N(0, 1)$

bit width = 1 bits

Design a quantisation function that
Minimises MSE of quantisation.

In ML, we are generally talking about a given Matrix of data.

$M =$

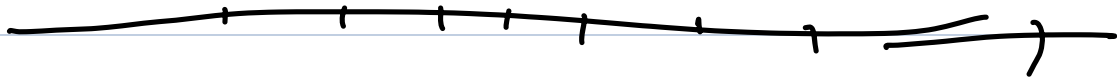
4.2	3.9	-1.5
-2.0	1.2	:
:	:	:
:	:	:

* Dynamic Quantisation: Assume M target data, tries to optimise Quantisation function just for that data

$\Rightarrow$ If this matrix was a Activation Matrix we need to re compute the Quantisation function.

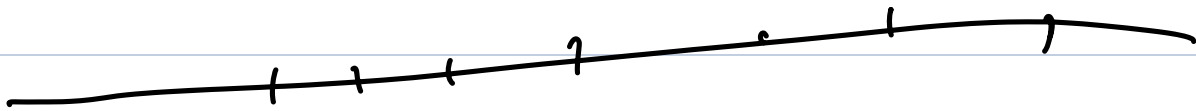
* Static Quantisation: Assume M to be representative of target distribution from which M was produced. Quantisation function is ~~the~~ computed once and reused again.

Q: Given a set of values (for ex. in a matrix), how do we quantise.

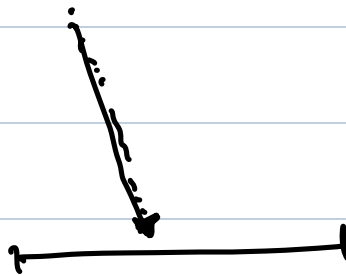


How do you design Quantisation function?

Things that we know



b bits $\rightarrow$
 $(2^b - 1)$
values.



How should we choose the function?
Really depends on the data.

"It would be nice if we could use
function and data is well approximated

with it"

linear. : why?

* Simple and only $O(1)$ storage

* lot of elementary operations can be done in quantised space

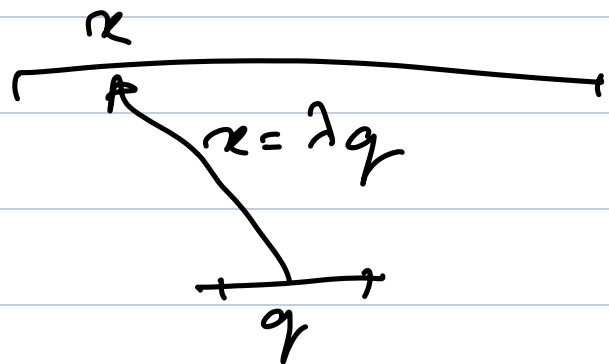
$$Q(x) = f(x)$$

$$Q(x_1 + x_2) = f(x_1) + f(x_2)$$

Why is this relevant?

Since, inner products are quite common in ML, we can do inner products in quantised space getting through per benefits.

Linear function:



* Ensure max is mapped to max of quantised.

$$r_{\max} = \max_i |x_i|$$

$$q_{\max} = 2^b - 1$$

$$r_{\max} = \lambda \cdot q_{\max}$$

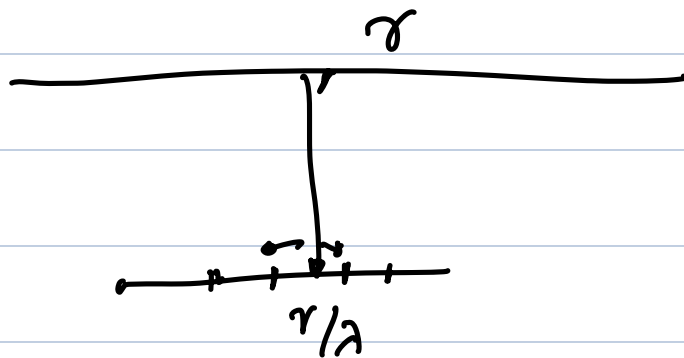
$$\lambda = r_{\max} / q_{\max}$$

function $r = \lambda q.$

$$\Phi^{-1}(q) = \lambda q.$$

$$\Phi(r) ?$$

$$\Phi(r) = \text{project}(r/\lambda)$$



Ex. Project = Round to nearest.

[Add Image linear.png]

Q. When would $r = \lambda q$ fail / be bad / be inefficient?

AI: $\hookrightarrow$ Distribution is shifted (Asymmetric)
 $\hookrightarrow \lambda q$ will waste a lot of bits in a region empty.
What should we do?

$\hookrightarrow$ Affine quantisation:

$$r = \lambda (q - z_q)$$

r : real, λ : scale

q : quantized z_q : quantised zero.

Two parameters: λ, z_q

How to find them, $r_{\max} \rightarrow q_{\max}$
 $r_{\min} \rightarrow q_{\min}$

$$r_{\max} = \lambda (q_{\max} - z_q)$$

$$r_{\min} = \lambda (q_{\min} - z_q)$$

$$\frac{I_{\max} - I_{\min}}{I_{\max} - I_{\min}} = 1$$

$$\checkmark \gamma = I_{\max} - \frac{I_{\max}}{1}$$

Zero point method

[Add Image affine-linear]

Q. when can linear fail.

↳ outliers? [Add Image clip1]

If outliers $\Rightarrow$ we do not get enough bit width to main chunk of data.

A. What to do:

① $\hookrightarrow$ Clipping : when is it okay?

↳ when outliers are not useful

[Add Image clip2, clip3]

(B) Rotations:

Outlier means that there is too much or few energy on a single Co-ordinates.

Q. What can we do to get rid of this?

↳ Distribute energy?

↳ Rotation??

Q. How do you choose the rotation matrix.

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.1 & 100 \end{bmatrix} \quad \text{rang} \approx 99.9$$

$$Hx = \begin{bmatrix} \frac{100.1}{\sqrt{2}} & \frac{99.9}{\sqrt{2}} \end{bmatrix} \quad \text{rang} = \frac{0.2}{\sqrt{2}}$$

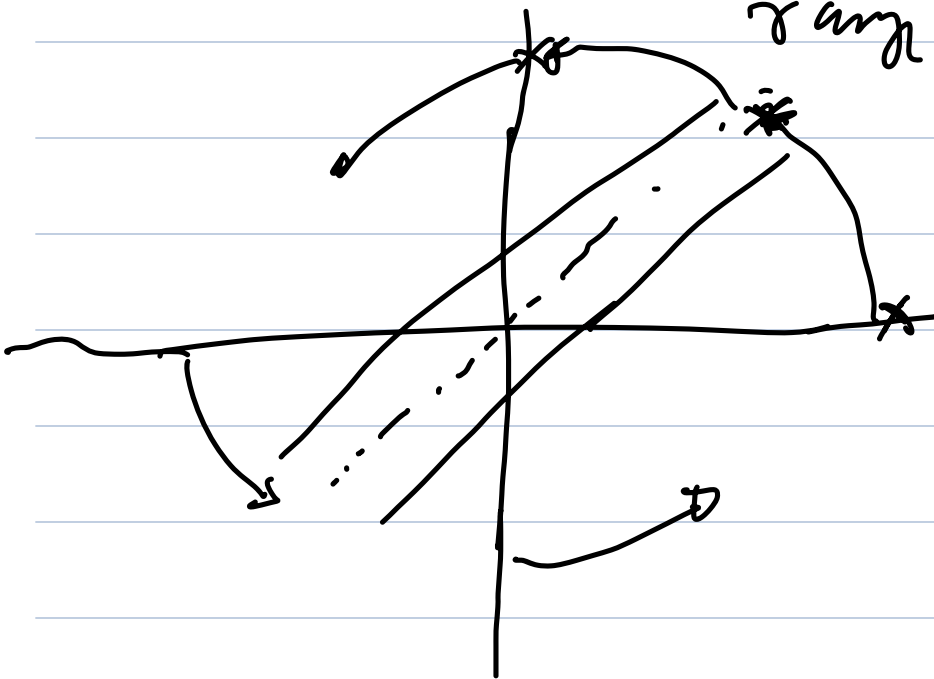
Consider Hadamard transform

What if we don't know x before hand.

$$x = \begin{bmatrix} 100 & 0.1 \end{bmatrix} \Rightarrow x = \begin{bmatrix} 0.1 & 100 \end{bmatrix}$$

$$Hx = \begin{bmatrix} \frac{100.1}{\sqrt{2}} & -\frac{99.9}{\sqrt{2}} \end{bmatrix}$$

$$\text{range} = \frac{200}{\sqrt{2}}$$



In fact, for any given rotation Matrix
 $\exists x$ where $\text{range}(x) < \text{range}(Hx)$

↳ Note that rotation always maintains the norms. $\|Hx\|_2 = \|x\|_2$

If we knew x before hand then we can "choose" or "learn" the rotation.

↳ But learning a $d \times d$ matrix for $x \in \mathbb{R}^d$

is not useful. So only if we can apply this matrix to a large number of x_i s (e.g. $X \in \mathbb{R}^{n \times d}$, $n \gg d$) then it's useful.

e.g.: If we observe that statistically some channel of an input to a DNN layer is "outlier", we can consider learning a rotation matrix which we will always apply.

Can we do something simpler.

↳ Consider "Random Rotations"

Consider $x \in \mathbb{R}^d$ $M = \begin{bmatrix} \end{bmatrix}$
 $d \times d$

For simplicity $M[i,j] = \text{i.i.d } \mathcal{N}(0, \frac{1}{\sqrt{d}})$

Let us look at

$$y = Mx$$

$$y_i = \sum_j M[i,j] x_j$$

$$y_i = \sum_j \frac{1}{\sqrt{d}} X_{ij} x_j \quad X_{ij} \sim \mathcal{N}(0,1)$$

$$E(y_i) = \frac{1}{\sqrt{d}} \cdot 0 \cdot x_j = 0$$

$$\text{Var}(y_i) = \sum_j \frac{x_j^2}{d} \cdot 1$$

$$= \frac{\|x\|_2^2}{d}$$

$$\text{std}(y_i) = \frac{\|x\|_2}{\sqrt{d}}$$

All co-ordinates are centered around 0 with std. $\frac{\|x\|_2}{\sqrt{d}}$

In fact $y_i \underset{\text{i.i.d.}}{\sim} \mathcal{N}\left(0, \frac{\|x\|_2}{\sqrt{d}}\right)$

$$E(\|y\|_2^2) = E(\sum y_i^2) \\ = d \cdot \frac{\|x\|_2^2}{d} = \|x\|_2^2$$

As $d \uparrow$ y_i concentrates around 0

Exercise:

$$\Pr(y_i > M) = ?$$

(1) Apply Chebyshev, Chernoff
and use the fact that
 $y_i \sim N(0, \frac{\|x\|_2^2}{d})$

Randomised Rotation: It being favorable
is an artifact of "high dimension"

Q. What other places can it fail?

→ Multi-Modal distribution of x

[Add. Image non-linear-quantisation pgs]

↳ Q. What can we do in such cases?

Code book approx

[Add weight-clustering png]

Code book gives flexibility to choose custom quantisation

But, dequantisation is now a lookup.

[Add Comparison table from slide 34]